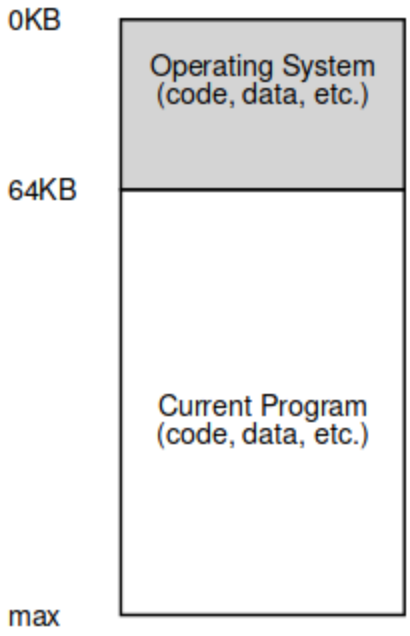
Абстракция: Адресное пространство

В ранние годы существования ОС, компьютерные системы были просты. Это происходило потому что пользователи не ожидали от систем многого. И именно эти гадкие пользователи с их ожиданиями простоты использования, высокой производительности, надежности привели к многим головным болям разработчиков.

Ранние системы

С точки зрения памяти ранние машины не предоставляли пользователям много абстрактности. В самом общем виде физическая память машины выглядела как-то вот так:



ОС тех дней была набором подпрограмм (реально как библиотека), которые располагались в памяти (начиная с физического адреса = 0 в этом примере) и помимо ОС в памяти будет всего один процесс (в данном примере располагается в памяти начиная с 64кб), который и будет использовать всю оставшуюся память. В этом примере мало иллюзий, создаваемых ОС и в таких системах пользователь и не ожидал многого. Жизнь разработчиков была куда проще.

Мультипрограммирование и разделение по времени

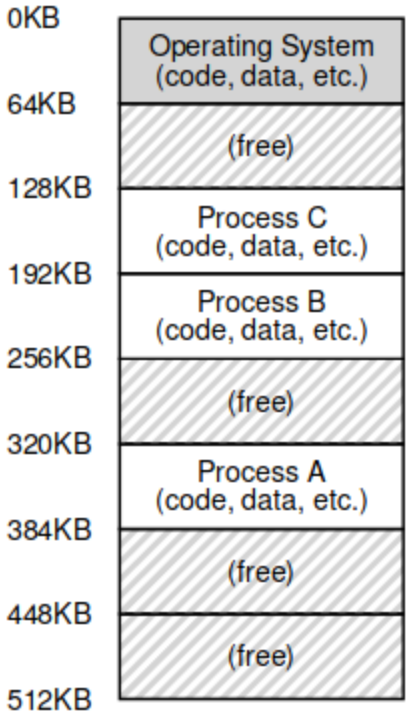
Через какое-то время из-за высокой стоимости машин люди начали использовать их совместно для повышения эффективности. Таким образом эра началась

мультипрограммирования, в которой несколько процессов готовы запустить работать и планировщик ОС должен решать как переключаться между ними, например для того чтобы дать поработать другим процессам, пока первый ожидает I/O. В те дни такое повышение эффективности было важным - машины стоили сотни тысяч долларов.

Вскоре, однако, люди начали требовать все больше и больше машин и началась эра разделения по времени. В частности многие усмотрели ограничения в пакетных вычислителях, а конкретно это затрагивало самих программистов, которых утомлял длинный и неэффективный процесс отладки. Понятие **интерактивность** стало важным, поскольку многие пользователи стали использовать машины одновременно, причем каждый ожидал (или надеялся) на ответ от их текущих исполняемых задач.

Один из способов воплощения разделения по времени - запускать один процесс на какое-то время, давая ему полный доступ к памяти, затем останавливать его, сохранять состояние на некоторый диск. Затем загружать в память состояние какого-либо другого процесса, который тоже будет работать некоторое время и таким образом реализуется разделение по времени.

К сожалению подобный подход вызывает некоторые проблемы: этот способ слишком медленный, в частности с ростом размера памяти. В то время как сохранение и восстановление состояния регистров довольно низко, то сохранение и восстановление всей памяти занимает уйму времени. И таким образом мы просто оставим процессы в памяти в момент переключения между ними, позволяя ОС эффективно заниматься разделением по времени.



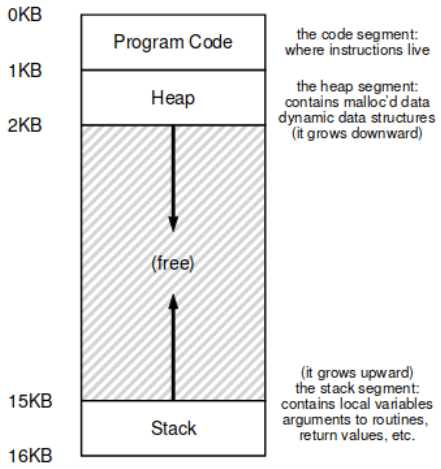
На рисунке изображены 3 процесса (A,B,C) и каждый из них использует небольшую часть физической памяти (512Кб). Предполагая использование одного CPU, ОС выбирает например процесс A, пока другие B,C ожидают в очереди своего запуска.

С повышением популярности разделения по времени к ОС стали предъявлять все больше и больше требований. В частности располагая несколько программ одновременно в памяти, повышается необходимость в защите, ведь мы не хотели бы позволять одному процессу читать или еще того хуже писать в чужую память.

Адресное пространство

ОС должна предоставлять пользователям простую в использовании абстракцию физической памяти. Такая абстракция называется адресным пространством и именно так программа видит память в системе. Понимание этого фундаментального момента является ключом в понимании того как память виртуализируется.

Адресное пространство процесса содержит всю информацию о запущенной программе. Например, код программы (инструкции) располагаются где-то в памяти и таким образом они находятся в его адресном пространстве. Программа во время своей работы использует стек для отслеживания того, где она находится в данный момент в цепочке вызова некоторой функции, а также для выделения локальных переменных и передаче параметров и значения кода возврата из подпрограммы. Наконец, куча используется для динамически выделяемой памяти, используемой пользователем, как например той, которую можно получить после вызова malloc() в языке C или new в объектно-ориентированных языках таких как JAVA или C++. Конечно помимо перечисленного существуют и многие другие вещи в памяти, например статически инициализированные переменные, но для простоты будем считать что есть всего 3 компонента: код, стек и куча.

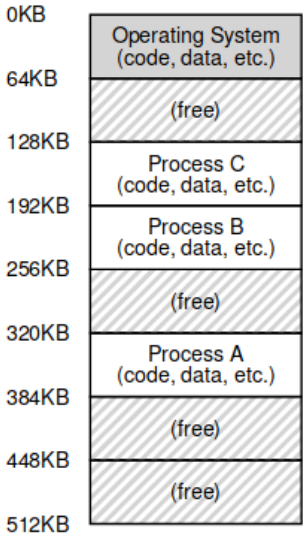


В данном примере будем рассматривать небольшое адресное пространство (16Кб). Код программы располагается на самом верху адресного пространства, начиная от 0 и заканчивая 1Кб. Код статичен и таким образом его проще всего поместить в память,

так что мы можем положить его на самый верх, зная, что он не потребует больше пространства по мере работы программы.

Далее, у нас есть еще 2 региона адресного пространства, которые могут расти и сокращаться по мере работы программы - это куча (наверху) и стек (внизу). Мы размещаем их следующим образом, потому что оба этих региона памяти желали бы расти и уменьшаться. Поместив их по разные стороны пространства мы таким образом даем им эту возможность роста - они растут в разные стороны. Куча начинается с 1К и растет вниз от меньших адресов к большим. Стек начинается с 16Кб и растет наверх, от больших адресов к меньшим. Однако, это расположение стека - всего лишь соглашение, расположить его в памяти можно в любом месте. И как мы увидим далее, когда несколько потоков сосуществуют в одном адресном пространстве, не существует какого-то правильного и удобного способа поделить адресное пространство как это показано сейчас.

Когда мы описываем адресное пространство, мы естественно говорим о неокторой форме абстракции, которую ОС предоставляет запущенной программе. ПРограмма естественно не находится в физической памяти по адресу 0-16Кб, на самом деле она загружается в некоторый произвольный участок физической памяти.



Как вы видите в данном примере, процессы располагаются в памяти в некторых случайный местах. Отсюда вытекает проблема:

Суть проблемы: как виртуализировать память
Как ОС может построить абстракцию приватного, возможно большого адресного пространства для нескольких одновременно запущенных процессов, используя одну физическую память?

Когда ОС делает это, мы говорим что она виртуализирует память, потому что запущенная программа думает, что загружена в память в какой-то конкретный адрес, например 0 и имеет потенциально огромное адресное пространство (32бита, 64бита), однако реальность выглядит несколько иначе.

Когда, например, процесс A пытается выполнить загрузку в адрес 0 (который мы называем **виртуальным адресом**), каким-то образом ОС с помощью поддержки оборудования должна убедиться, что вызов не идет в реальную физическую память с адресом 0, но наоборот идет в физическую память с адресом 320К (где A и загружен в памяти). Это и является ключом к виртуализации памяти, который лежит в основе каждой современной компьютерной системы.

Цели

ОС не только виртуализирует оперативную память, но выполняет над ней еще много других операций. Рассмотрим некоторые цели, которых нужно добиться для эффективного выполнения задач виртуализации.

Одна из главных целей системы виртуальной памяти - **прозрачность**. Прозрачность в данном контексте означает что сама система виртуализации незаметна для пользователя и программ. Таким образом программы не должны заботиться о том факте что память виртуализируется, напротив им следует вести себя так, будто у них есть своя собственная физическая память. ОС в свою очередь будет выполнять всю работу по распределению памяти между всеми процессами и поддерживать иллюзию.

Второй целью ВП (виртуальной памяти) является эффективность. ОС должна прилагать усилия и делать виртуализацию памяти наиболее эффективно, управляя временем (не замедляя работу программ) и пространством (не занимая чрезмерно много памяти). В воплощении эффективного менеджмента времени, ОС будет полагаться на поддержку оборудования, включая такие функции как TLB.

Третьей целью VM является защита. ОС должна убедиться, что процессы защищены друг от друга и самой ОС. Когда один процесс выполняет задачу загрузки, сохранения, получения следующей инструкции, он не должен иметь доступ или возможность влияния на память любого другого процесса и самой ОС, т.е. Не должен влиять ни на что за пределами адресного пространства. Таким образом мы можем говорить о термине **изоляция** в контексте памяти процессов. Каждый процесс должен быть запущен в собственном изолированном пространстве, которое защищено от других ошибочных или вредоносных процессов.